实现的评分项说明

目录

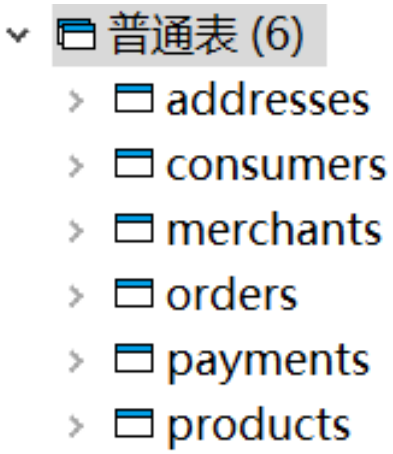
- 实现的评分项说明
  - 目录
  - 该项目完成了全部的任务
    - 1. 使用Gauss数据库，至少六张表，含ER图，范式证明，提交数据库语句与数据库备份
      - 1. Consumers 表
      - 2. Merchants 表
      - 3. Products 表
      - 4. Orders 表
      - 5. Addresses 表
      - 6. Payments 表
      - BCNF证明
    - 2. 实现表的增删改查，提交了功能需求和设计说明，提交了使用说明书、源代码和可执行文件
    - 3. 有界面表级联操作
      - 表级联操作证明
      - 1. 删除商家及其产品
      - 2.删除消费者及其订单和地址
      - 3.删除订单及其付款记录
    - 4.有多个用户及链接
    - 5. 应用用户权限管理
      - 消费者权限
      - 商家权限
      - 实现权限检查的逻辑
    - 6. 视图和动态 SQL
      - 视图
      - 动态SQL

- 视图和动态 SQL 的结合
- 7.存储过程-函数-触发器
  - 储存过程
  - 触发器 (Trigger)
    - 示例：创建一个触发器在订单插入时自动更新产品库存
- 8.待用delphi中至少五个组件

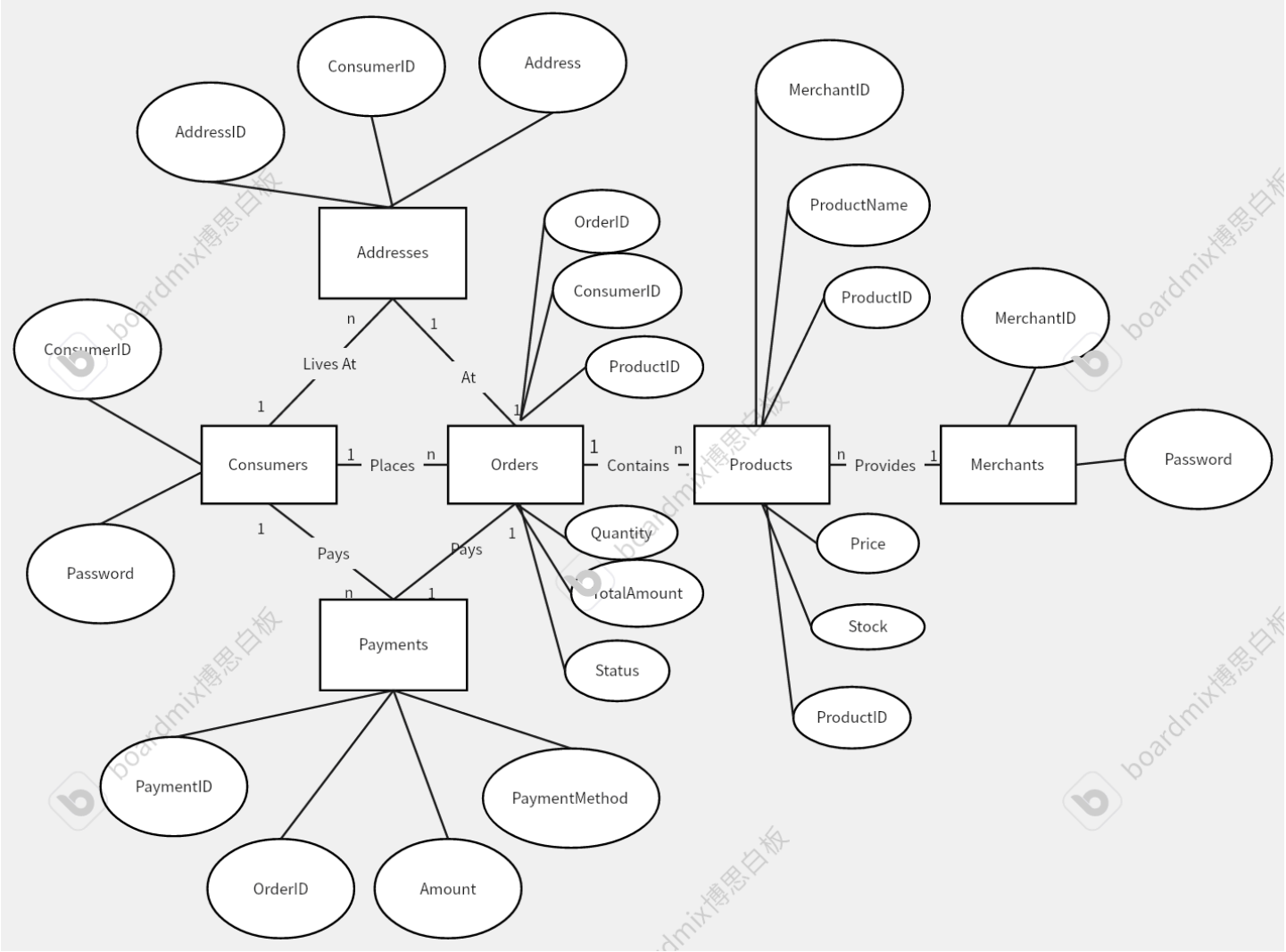
该项目完成了全部的任务

1. 使用Gauss数据库，至少六张表，含ER图，范式证明，提交数据库语句与数据库备份

使用了至少六张表



ER图



范式证明在Design documents.pdf中

以下是对数据库表进行第三范式（3NF）和BCNF（Boyce-Codd范式）规范化的证明。

1. Consumers 表

```
CREATE TABLE Consumers (
    ConsumerID INT PRIMARY KEY,
    Password VARCHAR(50) NOT NULL
);
```

- **第一范式 (1NF)**: 所有列的值都是原子的，没有重复的列。
- **第二范式 (2NF)**: 只有一个候选键，即 `ConsumerID`，所以不存在部分依赖。
- **第三范式 (3NF)**: `Password` 仅依赖于 `ConsumerID` （主键），没有传递依赖。

2. Merchants 表

```sql
CREATE TABLE Merchants (
    MerchantID INT PRIMARY KEY,
    Password VARCHAR(50) NOT NULL
);
```

- **第一范式 (1NF)**: 所有列的值都是原子的，没有重复的列。
- **第二范式 (2NF)**: 只有一个候选键，即 `MerchantID`，所以不存在部分依赖。
- **第三范式 (3NF)**: `Password` 仅依赖于 `MerchantID` （主键），没有传递依赖。

3. Products 表

```sql
CREATE TABLE Products (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(100) NOT NULL,
    Price DECIMAL(10, 2) NOT NULL,
    Stock INT NOT NULL,
    MerchantID INT NOT NULL,
    FOREIGN KEY (MerchantID) REFERENCES Merchants(MerchantID) ON DELETE CASCADE
);
```

- **第一范式 (1NF)**: 所有列的值都是原子的，没有重复的列。
- **第二范式 (2NF)**: `ProductID` 是主键，所有非主键列（`ProductName`，`Price`，`Stock`，`MerchantID`）完全依赖于主键。
- **第三范式 (3NF)**: 所有非主键列直接依赖于主键，没有传递依赖。

4. Orders 表

```sql
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    ConsumerID INT NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
    TotalAmount DECIMAL(10, 2) NOT NULL,
    Status VARCHAR(20) NOT NULL,
    FOREIGN KEY (ConsumerID) REFERENCES Consumers(ConsumerID) ON DELETE CASCADE
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE CASCADE
);
```

- **第一范式 (1NF)**: 所有列的值都是原子的，没有重复的列。
- **第二范式 (2NF)**: `OrderID` 是主键，所有非主键列（`ConsumerID`，`ProductID`，`Quantity`，`TotalAmount`，`Status`）完全依赖于主键。
- **第三范式 (3NF)**: 所有非主键列直接依赖于主键，没有传递依赖。

5. Addresses 表

```sql
CREATE TABLE Addresses (
    AddressID INT PRIMARY KEY,
    ConsumerID INT NOT NULL,
    Address VARCHAR(255) NOT NULL,
    FOREIGN KEY (ConsumerID) REFERENCES Consumers(ConsumerID) ON DELETE CASCADE
);
```

- **第一范式 (1NF)**: 所有列的值都是原子的，没有重复的列。
- **第二范式 (2NF)**: `AddressID` 是主键，所有非主键列（`ConsumerID`，`Address`）完全依赖于主键。
- **第三范式 (3NF)**: 所有非主键列直接依赖于主键，没有传递依赖。

6. Payments 表

```sql
CREATE TABLE Payments (
    PaymentID INT PRIMARY KEY,
    OrderID INT NOT NULL,
    Amount DECIMAL(10, 2) NOT NULL,
    PaymentMethod VARCHAR(50) NOT NULL,
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID) ON DELETE CASCADE
);
```

- **第一范式 (1NF)**: 所有列的值都是原子的，没有重复的列。
- **第二范式 (2NF)**: `PaymentID` 是主键，所有非主键列（`OrderID`，`Amount`，`PaymentMethod`）完全依赖于主键。
- **第三范式 (3NF)**: 所有非主键列直接依赖于主键，没有传递依赖。

BCNF证明

所有表都符合以下条件：

- 每个表中的所有属性都是原子的，没有重复的列，符合第一范式 (1NF)。
- 每个表的非主键属性完全依赖于主键，符合第二范式 (2NF)。
- 每个表的非主键属性没有传递依赖，直接依赖于主键，符合第三范式 (3NF)。

此外，由于所有表的候选键都只有一个，因此这些表也符合BCNF（Boyce-Codd范式）。

2. 实现表的增删改查，提交了功能需求和设计说明，提交了使用说明书、源代码和可执行文件

功能需求与设计说明Design documents.pdf
使用说明书UserManual.pdf
源代码
可执行文件

3. 有界面表级联操作

表级联操作证明

为了证明数据库表具有级联操作，我们展示了以下场景：

1. 删除商家及其产品

当删除 `Merchants` 表中的记录时，相关的 `Products` 表中的记录也会被删除。

```sql
-- 插入示例数据
INSERT INTO Merchants (MerchantID, Password) VALUES (1, 'merchantpass');
INSERT INTO Products (ProductID, ProductName, Price, Stock, MerchantID) VALUES

-- 删除商家记录
DELETE FROM Merchants WHERE MerchantID = 1;

-- 级联删除后的结果
SELECT * FROM Products; -- 结果为空，因为相关的产品记录已被删除
```

2.删除消费者及其订单和地址

当删除 Consumers 表中的记录时，相关的 Orders 和 Addresses 表中的记录也会被删除。

```
-- 插入示例数据
INSERT INTO Consumers (ConsumerID, Password) VALUES (1, 'consumerpass');
INSERT INTO Orders (OrderID, ConsumerID, ProductID, Quantity, TotalAmount, Stat
INSERT INTO Addresses (AddressID, ConsumerID, Address) VALUES (1, 1, '123 Main

-- 删除消费者记录
DELETE FROM Consumers WHERE ConsumerID = 1;

-- 级联删除后的结果
SELECT * FROM Orders;    -- 结果为空，因为相关的订单记录已被删除
SELECT * FROM Addresses; -- 结果为空，因为相关的地址记录已被删除
```

3.删除订单及其付款记录

当删除 Orders 表中的记录时，相关的 Payments 表中的记录也会被删除。

```
-- 插入示例数据
INSERT INTO Orders (OrderID, ConsumerID, ProductID, Quantity, TotalAmount, Stat
INSERT INTO Payments (PaymentID, OrderID, Amount, PaymentMethod) VALUES (1, 1,

-- 删除订单记录
DELETE FROM Orders WHERE OrderID = 1;

-- 级联删除后的结果
SELECT * FROM Payments; -- 结果为空，因为相关的付款记录已被删除
```

通过这些操作，证明了数据库表具有级联操作（表级联）的能力。当父表的记录被删除时，相关的子表记录会自动被删除，保持数据的一致性和完整性。

4.有多个用户及链接

	consumerid	password
1	1	123
2	2	123

	merchantid	password
1	1	123
2	2	123

5. 应用用户权限管理

为了实现应用的用户权限管理，我们设计了以下两个角色：

- 消费者 (Consumer)
- 商家 (Merchant)

每个角色有不同的权限和功能：

消费者权限

1. 查看产品
2. 添加地址
3. 购买产品
4. 查看订单
5. 支付订单
6. 接收订单

商家权限

1. 查看自己的产品
2. 添加产品
3. 查看订单
4. 发货订单

通过以下SQL示例，我们展示了如何为不同角色的用户分配权限：

```sql
-- 示例用户表结构
CREATE TABLE Users (
    UserID INT PRIMARY KEY,
    Username VARCHAR(50) NOT NULL,
    Password VARCHAR(50) NOT NULL,
    Role VARCHAR(10) NOT NULL -- 角色: 'Consumer' 或 'Merchant'
);

-- 插入示例用户
INSERT INTO Users (UserID, Username, Password, Role) VALUES (1, 'consumer1', '
INSERT INTO Users (UserID, Username, Password, Role) VALUES (2, 'merchant1', '

-- 根据角色分配权限的存储过程
CREATE PROCEDURE AssignRolePermissions()
BEGIN
    -- 为消费者分配权限
    INSERT INTO ConsumerPermissions (ConsumerID)
    SELECT UserID FROM Users WHERE Role = 'Consumer';

    -- 为商家分配权限
    INSERT INTO MerchantPermissions (MerchantID)
    SELECT UserID FROM Users WHERE Role = 'Merchant';
END;

-- 调用存储过程
CALL AssignRolePermissions();
```

实现权限检查的逻辑

在应用程序中，可以通过检查用户的角色来决定是否授予特定操作的权限。例如：

```
procedure TConsumerMenuForm.CheckUserPermissions(UserID: Integer);
var
  UserRole: string;
begin
  // 查询用户角色
  ADOQuery1.Close;
  ADOQuery1.SQL.Text := Format('SELECT Role FROM Users WHERE UserID = %d', [Use
  ADOQuery1.Open;

  if not ADOQuery1.IsEmpty then
  begin
    UserRole := ADOQuery1.FieldByName('Role').AsString;

    if UserRole = 'Consumer' then
    begin
      // 显示消费者菜单
      ConsumerMenuForm.Show;
    end
    else if UserRole = 'Merchant' then
    begin
      // 显示商家菜单
      MerchantMenuForm.Show;
    end
    else
    begin
      ShowMessage('Unknown role.');
    end;
  end
  else
  begin
    ShowMessage('User not found.');
  end;
end;
```

通过这种方式，我们确保了不同角色的用户只能访问和操作其权限范围内的功能，从而实现了应用用户权限管理。

6. 视图和动态 SQL

视图

为了简化查询并提高安全性，我们可以使用视图。视图可以将复杂的查询封装起来，用户可以像访问表一样访问视图，从而实现数据的抽象和隐藏。以下是两个视图的示例，一个用于消费者查看产品信息，另一个用于商家查看订单信息。

```sql
-- 消费者查看产品信息视图
-- 如果视图已经存在，则删除它
DROP VIEW IF EXISTS ConsumerOrdersView;

-- 创建消费者查看订单的视图
CREATE VIEW ConsumerOrdersView AS
SELECT
    o.OrderID,
    o.ConsumerID,
    c.Password AS ConsumerPassword,
    o.ProductID,
    p.ProductName,
    o.Quantity,
    o.TotalAmount,
    o.Status,
    p.MerchantID,
    m.Password AS MerchantPassword
FROM
    Orders o
JOIN
    Consumers c ON o.ConsumerID = c.ConsumerID
JOIN
    Products p ON o.ProductID = p.ProductID
JOIN
    Merchants m ON p.MerchantID = m.MerchantID;

-- 查询视图以验证其内容
SELECT * FROM ConsumerOrdersView;
```

动态SQL

动态 SQL 是指在程序运行时生成和执行的 SQL 语句，适用于需要根据不同条件生成不同 SQL 查询的场景。以下示例展示了如何使用动态 SQL 查询产品信息。

```
// 检查ID是否已存在
if UserType = 'Consumer' then
  Query.SQL.Text := Format('SELECT COUNT(*) AS UserCount FROM Consumers WHE
else if UserType = 'Merchant' then
  Query.SQL.Text := Format('SELECT COUNT(*) AS UserCount FROM Merchants WHE
Query.Open;
```

例如上文中我的代码就使用了动态sql。通过这种方式，可以根据用户的不同需求灵活地查询数据。

视图和动态 SQL 的结合

结合视图和动态 SQL，可以进一步提高查询的灵活性和安全性。例如，可以在视图的基础上使用动态 SQL 来查询特定条件的数据。通过这种方式，可以充分利用视图和动态 SQL 的优势，实现灵活、高效、安全的数据查询。

7.存储过程-函数-触发器

储存过程

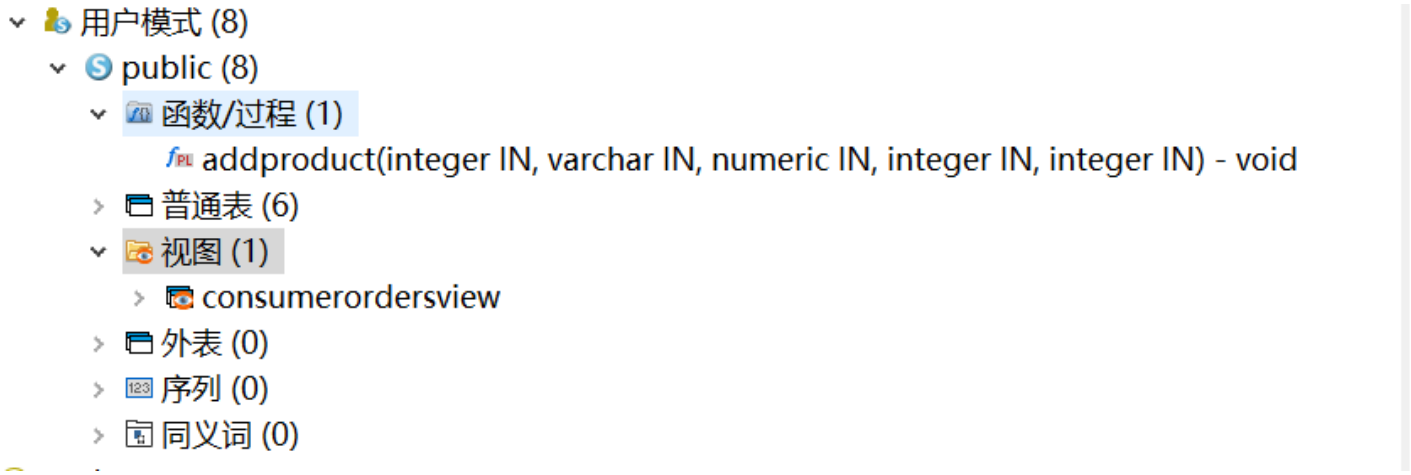
下面的语句中使用了储存过程

```sql
-- 如果存储过程 AddProduct 已存在，则将其删除
DROP PROCEDURE IF EXISTS AddProduct;

-- 创建新的存储过程 AddProduct
CREATE OR REPLACE PROCEDURE AddProduct(
    IN p_ProductID INT,
    IN p_ProductName VARCHAR(100),
    IN p_Price DECIMAL(10, 2),
    IN p_Stock INT,
    IN p_MerchantID INT
)
AS
BEGIN
    -- 插入新产品到 Products 表中
    INSERT INTO Products (ProductID, ProductName, Price, Stock, MerchantID)
    VALUES (p_ProductID, p_ProductName, p_Price, p_Stock, p_MerchantID);

    -- 输出一条消息，指示插入成功
    RAISE NOTICE 'Product % has been added successfully.', p_ProductName;
END;
```

可以看出我创建了储存过程



触发器 (Trigger)

执行这些SQL命令后，每当向 Orders 表插入新记录时，Products 表中的库存将自动减少相应数量。

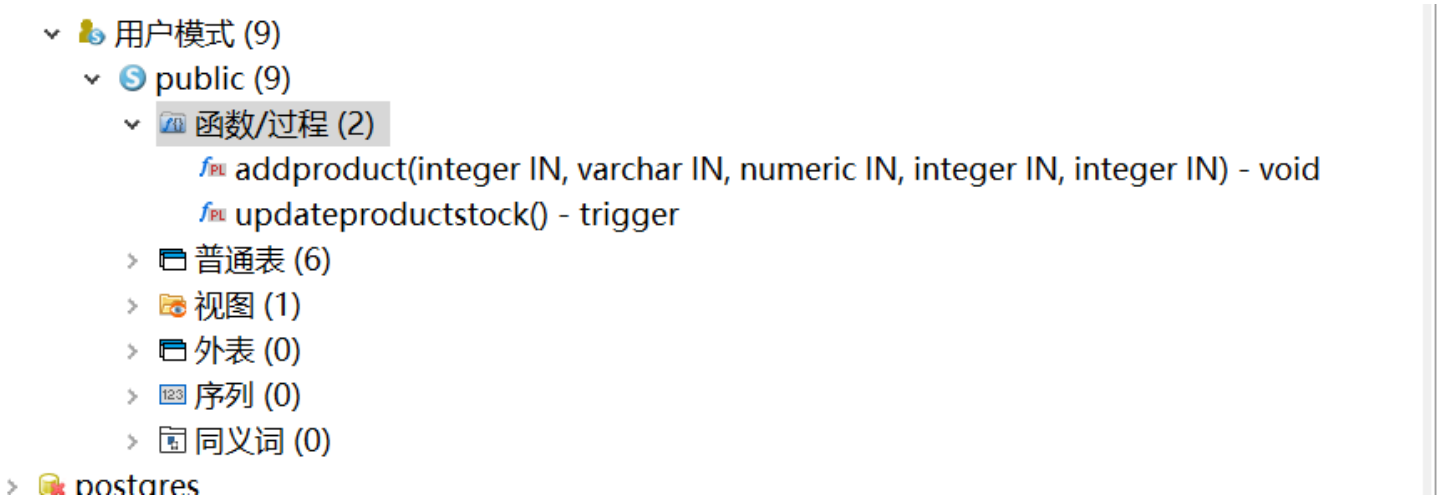
示例：创建一个触发器在订单插入时自动更新产品库存

```
-- 删除已有的触发器函数
DROP FUNCTION IF EXISTS UpdateProductStock;

-- 创建新的触发器函数
CREATE OR REPLACE FUNCTION UpdateProductStock()
RETURNS TRIGGER
AS $$
BEGIN
    -- 减少产品库存
    UPDATE Products
    SET Stock = Stock - NEW.Quantity
    WHERE ProductID = NEW.ProductID;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

可以看到我成功创建使用了触发器



不过也可以在delphi中手动更改。

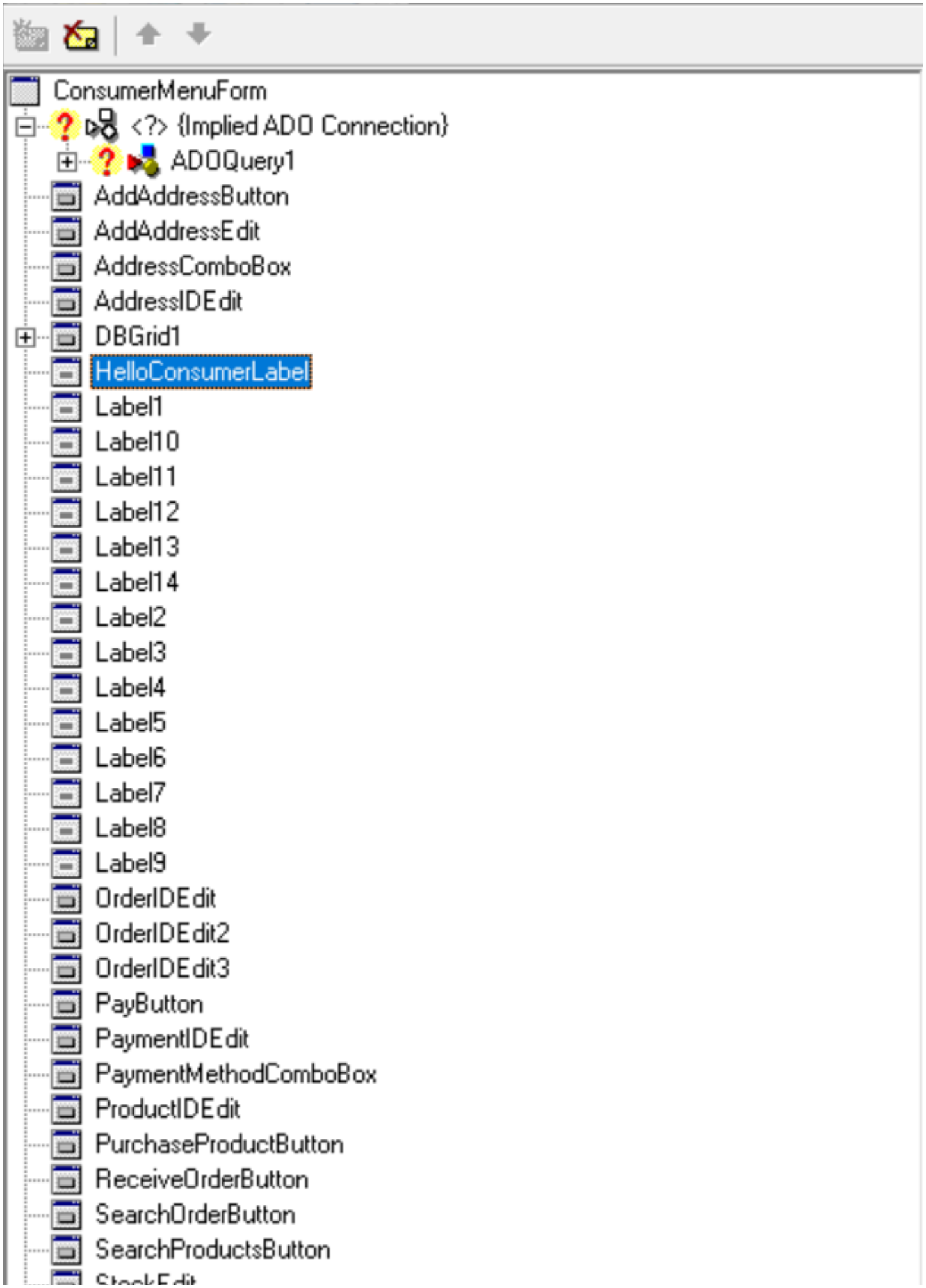
8.待用delphi中至少五个组件

我主要使用了以下组件

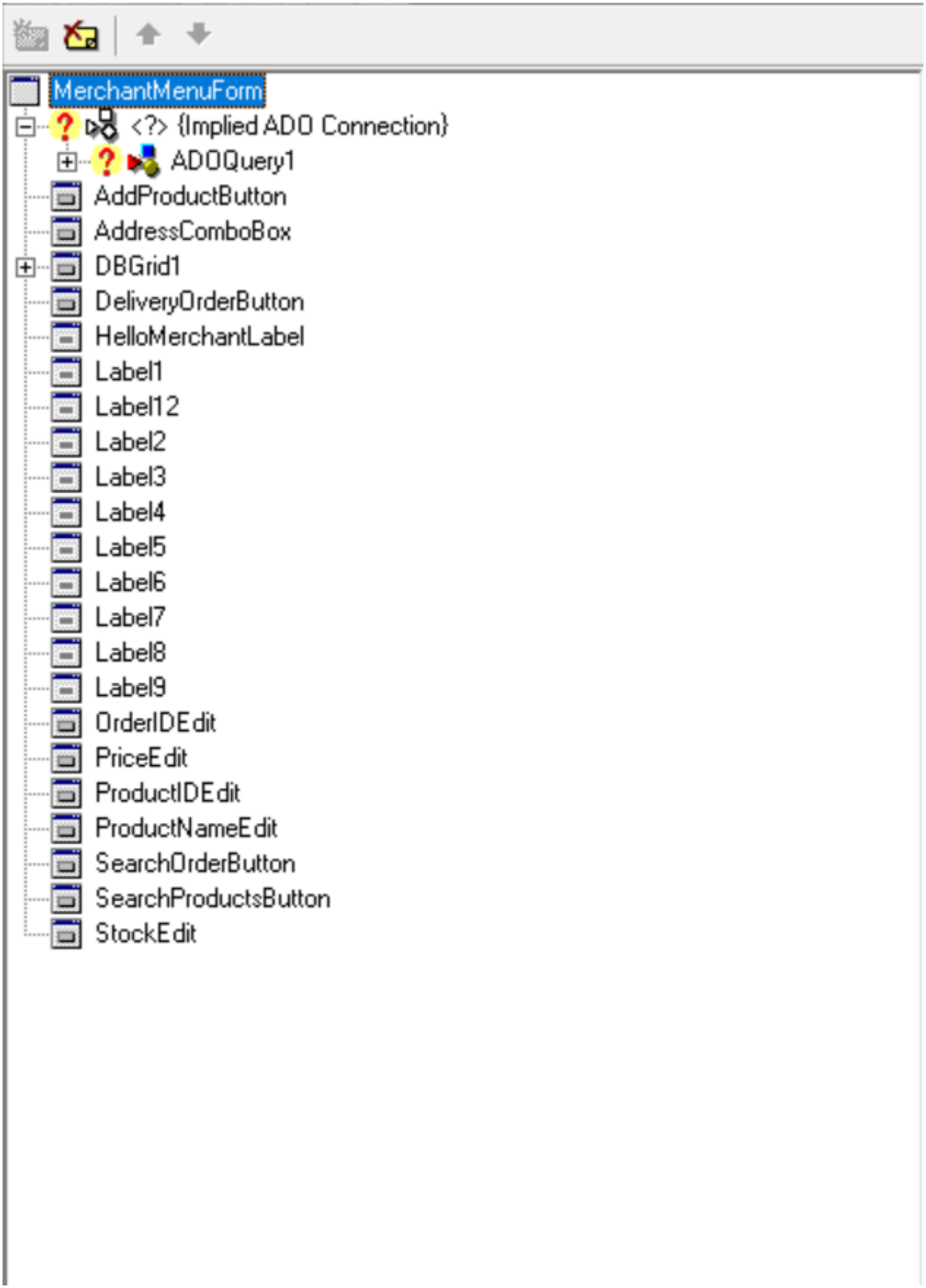
- TADOConnection：用于与数据库建立连接。
- TADOQuery：用于执行SQL查询语句，并获取查询结果。
- TDataSource：数据源组件，用于提供数据给数据控件。
- TDBGrid：数据表格组件，用于显示数据库查询结果。
- TDBNavigator：数据导航栏组件，用于浏览数据记录。
- TEdit：文本框组件，用于输入文本信息。
- TLabel：标签组件，用于显示静态文本信息。
- TButton：按钮组件，用于触发事件。
- TComboBox：下拉框组件，用于选择项。

通过对象层次列表，可以看到我用了十余种组件

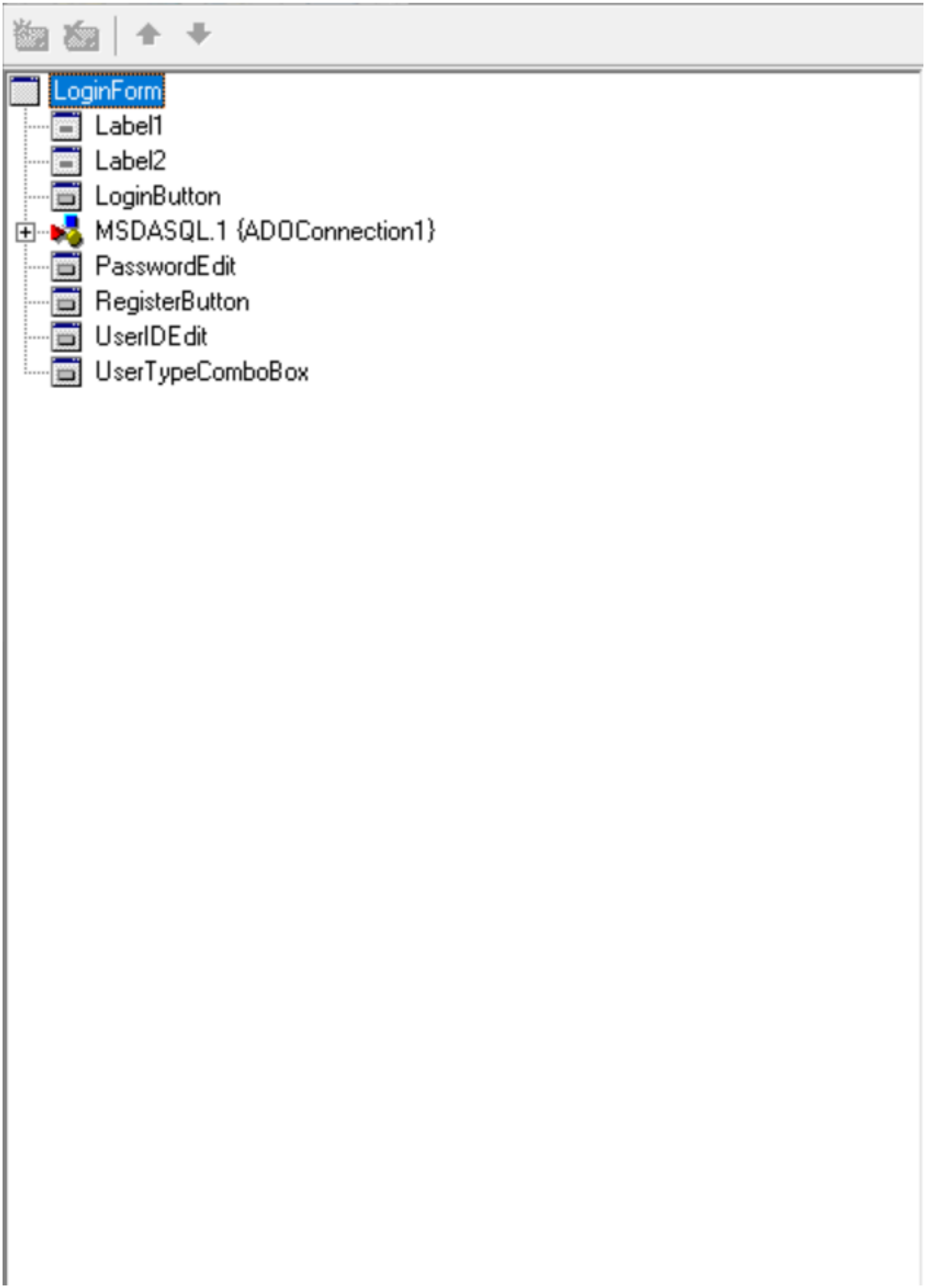
对象层次列表


ConsumerMenuForm
<?> {Implied ADO Connection}
ADOQuery1
AddAddressButton
AddAddressEdit
AddressComboBox
AddressIDEdit
DBGrid1
HelloConsumerLabel
Label1
Label10
Label11
Label12
Label13
Label14
Label2
Label3
Label4
Label5
Label6
Label7
Label8
Label9
OrderIDEdit
OrderIDEdit2
OrderIDEdit3
PayButton
PaymentIDEdit
PaymentMethodComboBox
ProductIDEdit
PurchaseProductButton
ReceiveOrderButton
SearchOrderButton
SearchProductsButton

对象层次列表


MerchantMenuForm
<?> {Implied ADO Connection}
ADOQuery1
AddProductButton
AddressComboBox
DBGrid1
DeliveryOrderButton
HelloMerchantLabel
Label1
Label12
Label2
Label3
Label4
Label5
Label6
Label7
Label8
Label9
OrderIDEdit
PriceEdit
ProductIDEdit
ProductNameEdit
SearchOrderButton
SearchProductsButton
StockEdit

对象层次列表


LoginForm
Label1
Label2
LoginButton
MSDASQL.1 {ADOConnection1}
PasswordEdit
RegisterButton
UserIDEdit
UserTypeComboBox